

Definitions of Turing machines

* Turing Machine:

Turing Machine was invented in 1936 by Alan O. Turing. It is an accepting device which accepts recursive Enumerable language generated by type 0 grammar.

Features of TM:

- ① It has external memory which remembers arbitrary long sequence of i/p.
- ② It has unlimited memory capability.
- ③ The model has a facility by which the i/p at left or right on the tape can be easily read.
- ④ Machine can produce certain output based on its i/p.

TM is a mathematical model which consists infinite length tape divided into cells on which i/p is given. It consists head which reads the i/p tape. A state register stores the state of the Turing M/c. After reading i/p symbol it is replaced with another symbol, its internal state is change, it moves from one cell to right or left. TM reaches to final state, the i/p string is accepted, otherwise it is rejected.

M T W T F S S
Page No.:
Date:
YOUVA

A TM can be described as a 7-tuple
 $(Q, X, \Sigma, \delta, q_0, B, F)$ where,

Q - is a finite set of state.

X - is the tape alphabet

Σ - is input alphabet

δ - is a transition function $\delta: Q \times X \rightarrow Q \times X$
left shift, Right shift

q_0 - is the initial state.

B - is the blank symbol

F - is final state.

eg:

* Representation of Turing Machine:

The specification of Turing Machine means to give a description of 7-tuple in general delination. The different representation are explained on two particular Turing Machine.

1) Look up table Representation

2) Graph representation

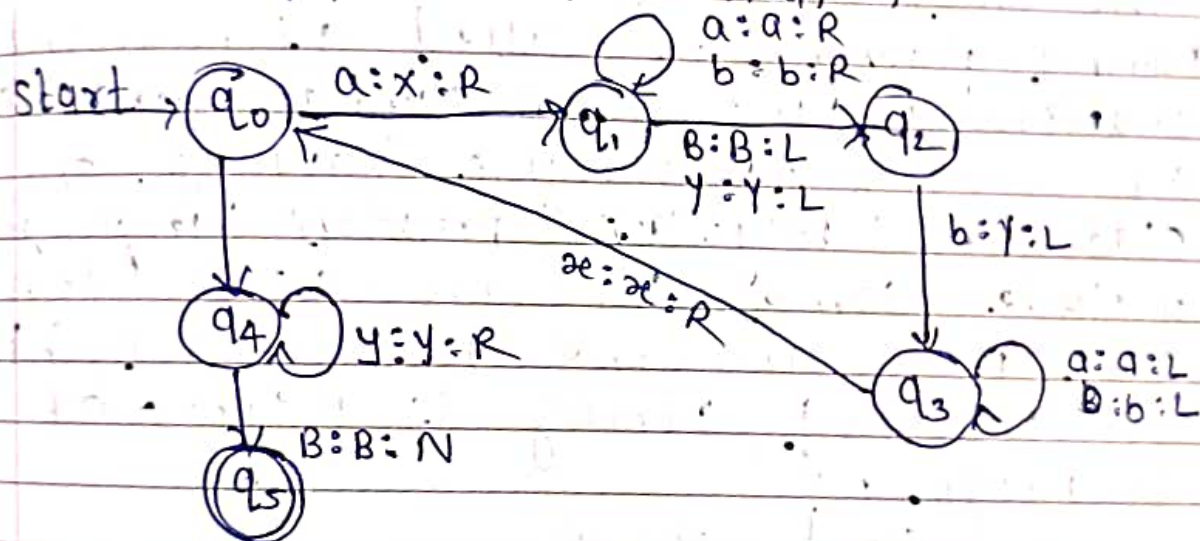
3) Representing TM by enumeration.

eg: Turing Mlc For $anbn$, 1

1) look up table Representation of TM
S is mapped as

Q/S	a	b	x	y	TS
$\rightarrow q_0$	(q_1, xR)	-	-	(q_2, yR)	-
q_1	(q_1, xR)	(q_1, bR)	-	(q_2, yR)	(q_2, b, L)
q_2	-	$(q_3, y, 2)$	-	-	-
q_3	(q_3, q_1, L)	(q_3, b, L)	(q_0, x, R)	-	-
q_4	-	-	-	(q_4, y, R)	(q_5, b, N)
q_5	-	-	-	-	-

2) Graphical Representation



3) Representation of TM by enumeration

an enumerator is like a python generator. It constantly enumerates words. There are several ways to define this formally, but suppose that Turing machine has a special o/p tape, a special o/p state. The word on the o/p tape is o/p and TM continues its operation in post-o/p state. In this TM can o/p many words, possibly infinitely many. In order to enumerate.

these words from your language L , it can
 $L = \{a^n b^n \mid n \geq 1\}$

- o/p of the empty word.
- o/p of the word ab
- o/p of the word $aabb$

the machine eventually o/p every word of the form $a^n b^n$

* language acceptability of TM :-

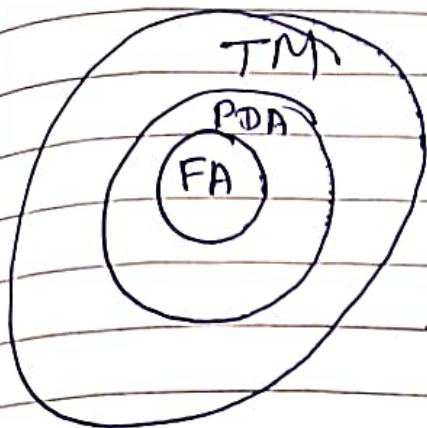
Recursive enumerable & Recursive language:

The class of language accepted/recognised by Turing machine is called as recursively enumerable language or Turing acceptable or Turing recognizable language. The class of Regular expression language include some language for which we can not answer the question, whether an arbitrary string is member of language,

That means,

If M is a TM. accepting a such language, then M must fail to halt on some I/P. If the I/P w is acceptable then M will eventually halt on w . But if w is not acceptable, M continuously Run forever.

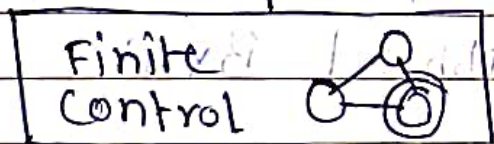
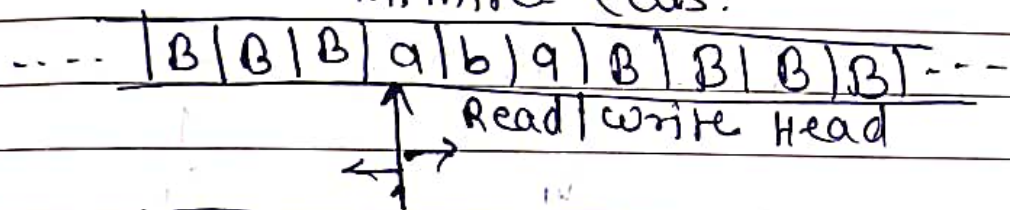
* Formal Definition of Turing Machine



- TM can accept the language like $a^n b^n c^n$ i.e. it is possible to perform no. of comparison in Turing MLC.
- TM is more powerful than FA and PDA.

Regular language [RL] = FA
 context free language [CFL] = PDA
 Recursively enumerable language - TM.

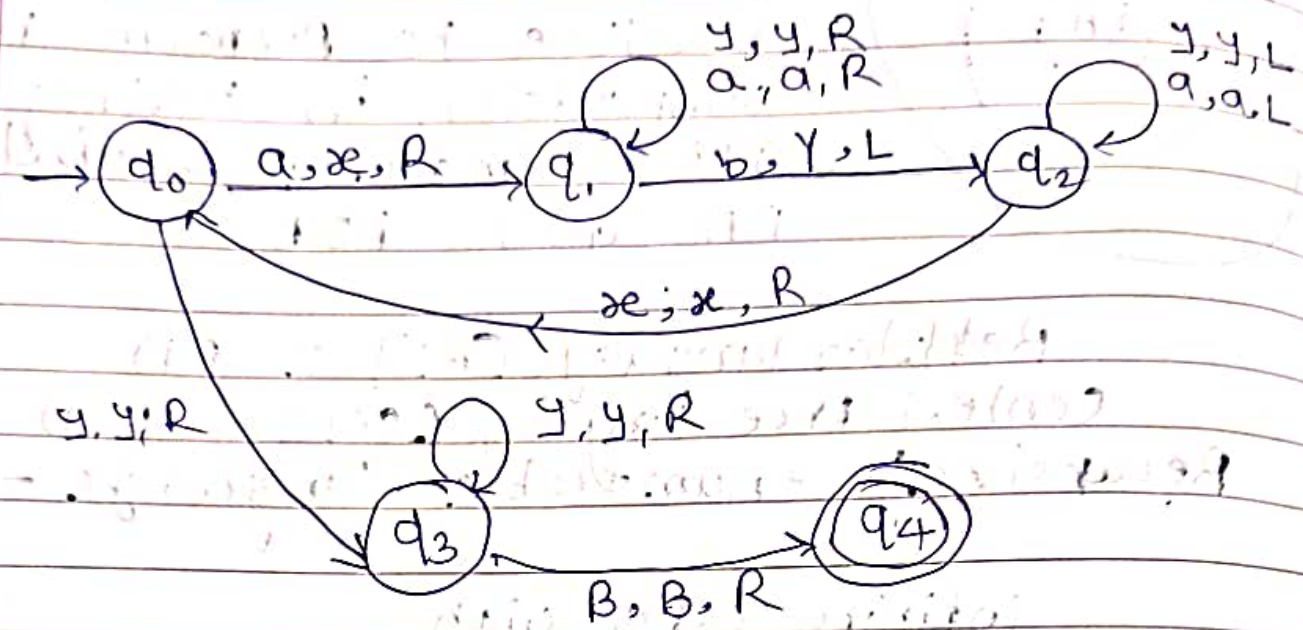
infinite tape with infinite cells.



operation by head

- 1) Read/scan by Head
- 2) write/update by head
- 3) Move head to Left/Right.

eg: Design a Turing machine for $a^n b^n$ | $n \geq 1$
 $L = \{ab, aabb, aaabbb, \dots\}$



$(q_0, aaabbb) \vdash Bx a' b b B B \vdash B x a' b b B B$
↑ ↑
 q_1 q_1

$\vdash B x a b b B B \vdash B x a b b B \vdash B x a b b$
↑ ↑ ↑
 q_1 q_1 q_2

$\vdash B x a b y B \vdash B x a b y B \vdash B x a b y B$
↑ ↑ ↑
 q_3 q_3 q_3

$\vdash B x x b y B \vdash B x x b y B \vdash B x x b y B$
↑ ↑ ↑
 q_1 q_1 q_2

$\vdash B x x y y B \vdash B x x y y B \vdash B x x y y B$
↑ ↑ ↑
 q_3 q_0 q_4

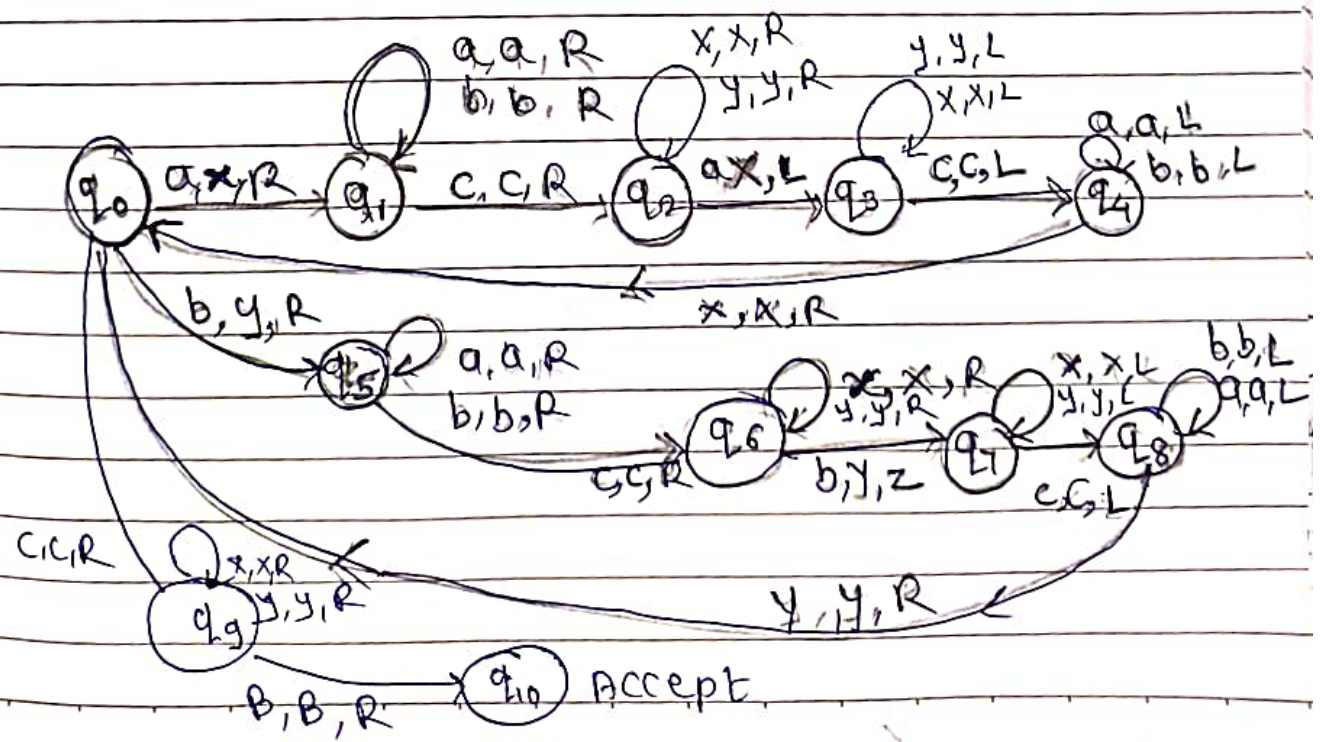
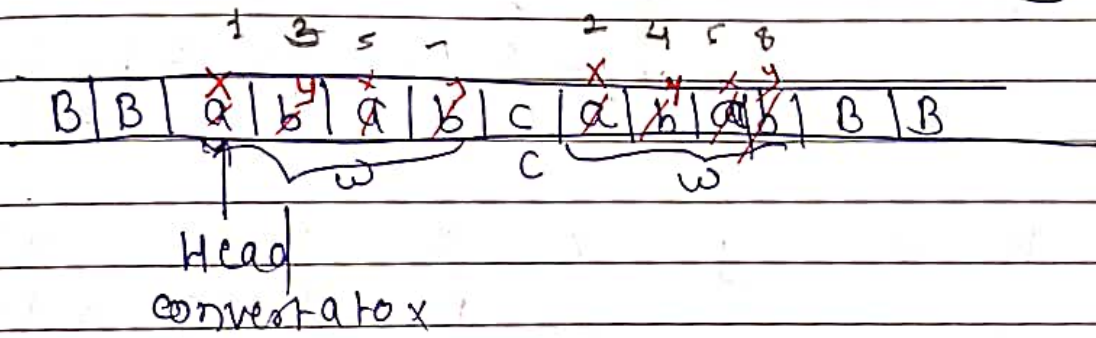
$\vdash B x x y y B \vdash \text{Halt}$
↑

Transition Table

Q \ E	a	b	x	y	B
→ q ₀	(q ₁ , x, R)	-	-	(q ₄ , y, R)	-
q ₁	(q ₁ , a, R)	(q ₂ , b, R)	-	(q ₀ , y, L)	(q ₄ , B, L)
q ₂	-	(q ₃ , y, L)	-	-	-
q ₃	(q ₃ , y, L)	(q ₃ , y, L)	(q ₀ , x, R)	-	-
q ₄	-	-	-	(q ₄ , y, R)	(q ₅ , B, R)
Ⓠ q ₅	-	-	-	-	-

Q: Design Turing Machine for
 $w \in W \mid w \in \{a, b\}^*$

$L = \{acb, abab, cabab, abcab, \dots\}$



~~B a b a b c b a b a B~~
~~↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑~~

B ^xa b a b c ^xa b a b B
 ↑ → ↑ → ↑ → ↑ → ↑ → ↑
 ↑ ← ↑ ← ↑ ← ↑ ← ↑ ←

B ^yx b a b c x ^yb a b B
 ↑ → ↑ → ↑ → ↑ → ↑ → ↑ → ↑
 ↑ ← ↑ ← ↑ ← ↑ ← ↑ ←

B x ^xy o b c x ^xy a b B
 ↑ → ↑ → ↑ → ↑ → ↑ → ↑ → ↑
 ↑ ← ↑ ← ↑ ← ↑ ← ↑ ←

B x y x ^yb c x y x ^yb B
 ↑ → ↑ → ↑ → ↑ → ↑ → ↑ → ↑
 ↑ ← ↑ ← ↑ ← ↑ ← ↑ ←

B x y x y c x y x y
 ↑ → ↑ → ↑ → ↑ → ↑ → ↑ → ↑

eg: Design Turing Machine

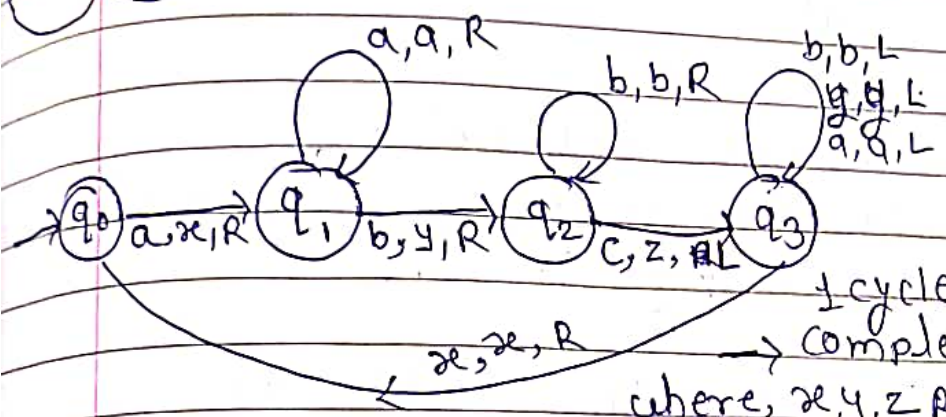
for $\{a^n b^n c^n \mid n \geq 1\}$

$\{abc, aabbcc, aaabbbccc, \dots\}$

... B | a | a | b | b | b | c | c | c | B | ...

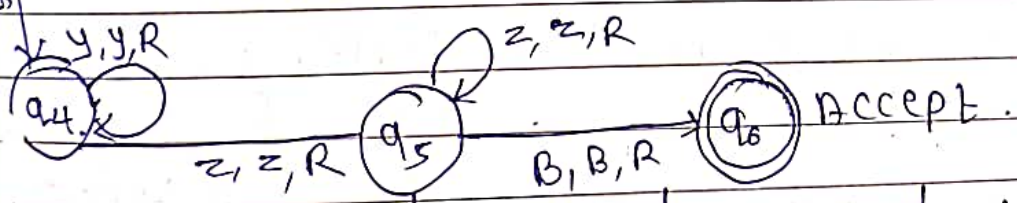
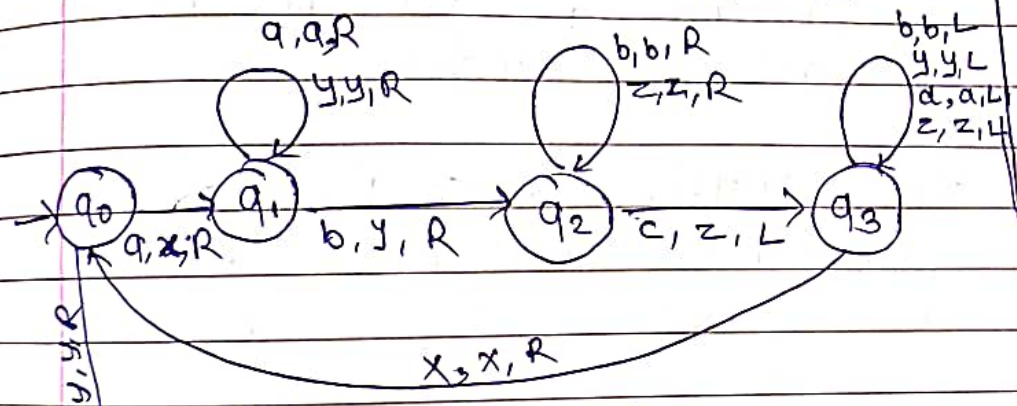
$\begin{matrix} a \rightarrow x \\ b \rightarrow y \\ c \rightarrow z \end{matrix}$ Replace

consider string 'aaabbbccc'



↓ cycle completed
 where, x, y, z Replace one abc.

B^xa^yaabb^zbbccc^xB
 ↑↑↑↑↑↑↑↑
 ↑↑↑↑↑↑↑↑
 B^xx^xa^yy^yb^zzzc^xB
 ↑↑↑↑↑↑↑↑
 ↑↑↑↑↑↑↑↑
 B^xx^xx^xy^yy^yz^zzz^zB
 ↑↑↑↑↑↑↑↑



	a	b	c	x	y	z	B
q ₀	(q ₀ , x, R)	—	—	—	(q ₄ , y, R)	—	—
q ₁	(q ₁ , a, R)	(q ₂ , y, R)	—	—	(q ₁ , y, R)	—	—
q ₂	—	(q ₂ , b, R)	(q ₃ , z, L)	—	—	(q ₂ , z, R)	—
q ₃	(q ₃ , a, L)	(q ₃ , b, L)	—	(q ₀ , x, R)	(q ₃ , y, L)	(q ₃ , z, L)	—
q ₄	—	—	—	—	(q ₄ , y, R)	(q ₄ , z, R)	(q ₅ , B, R)

Halting problem.

Imp Halting problem is undecidable.

asking

- is it possible to tell whether a m/c will halt in some given i/p?

eg: input: A TM & i/p string w .

problem

Does the TM finish computing to the string w in a finite no. of steps?

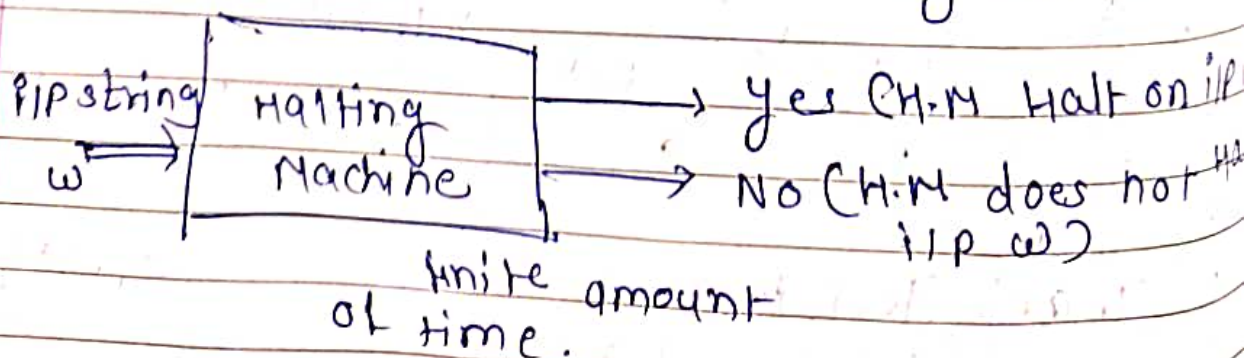
proof:

Assume TM exists to solve this problem & then we will get it is contradicting itself.

- we will call this T.M. as a Halting m/c that produce a 'yes' or 'no' in finite amount of time.

- If the Halting m/c finishes in a finite amount of time the output comes as 'yes' otherwise as 'no'.

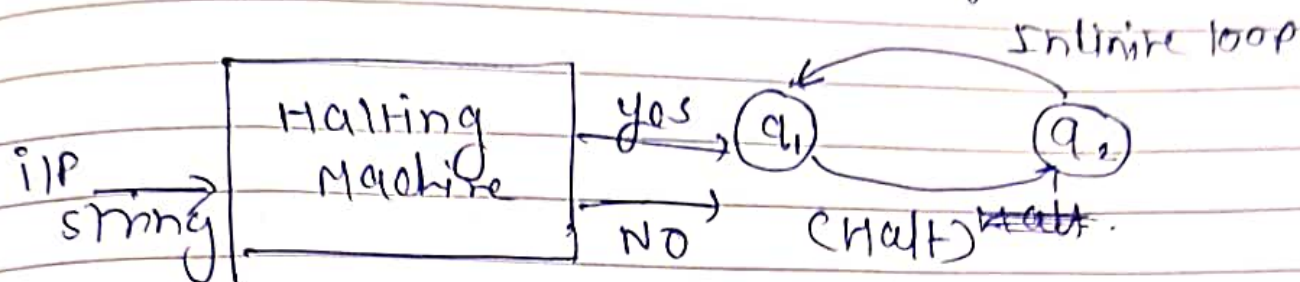
- The block dia of Halting m/c:



Now we will design an inverted Halting Machine as

- if H Returns yes then loop forever
- if H returns no, then Halt.

The idea of inverted Halting M/C :-



after that a m/c $(HM)_2$ which input itself constructed as -

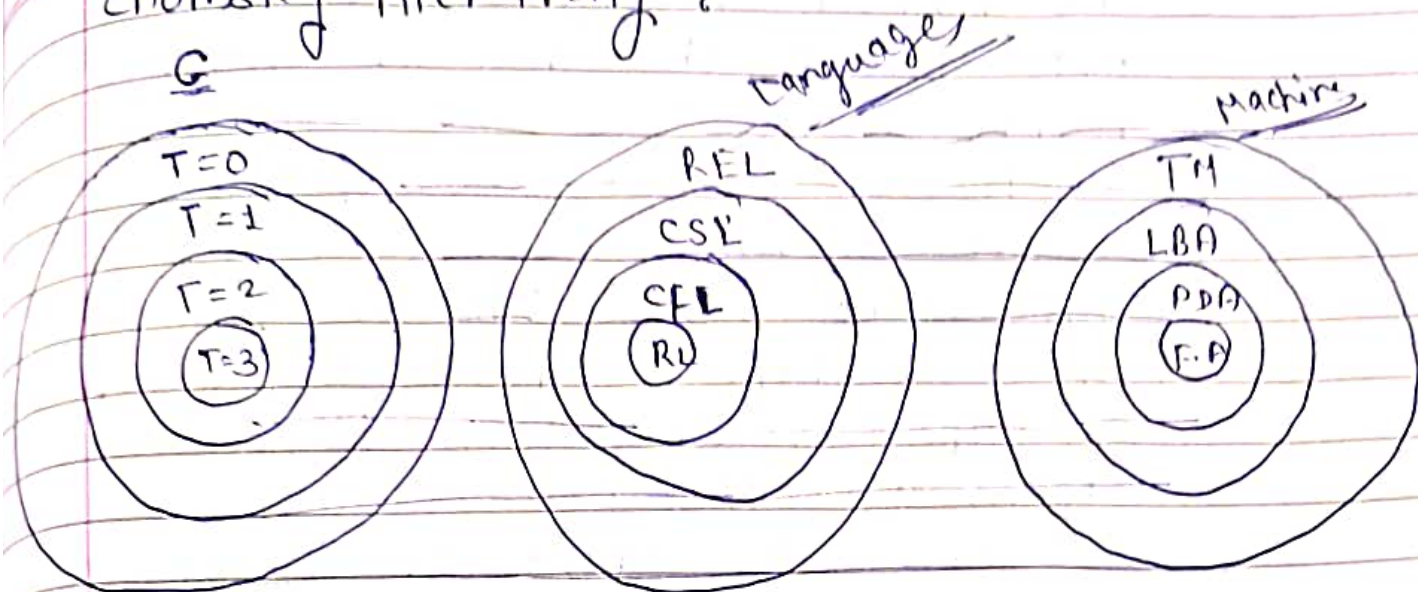
- if $(HM)_2$ Halts on i/p loop forever
- else Halt.

After this we got a Contradiction then the Halting prob is undecidable.

loop Halt (n, w)

```
{
  if " will halt  $(m, w)$  then
  {
    while(True);
  }
  else
    return false;
}
```

Chomsky Hierarchy :



Chomsky Hierarchy represents the class of languages that are accepted by the different machine. The category of language in Chomsky's Hierarchy is

① Type 0 :

Type 0 is known as unrestricted grammar. There is no restriction on the grammar rules of these types of languages. These languages can be efficiently modeled by TM.

eg : $bAa \rightarrow aa$
 $S \rightarrow S$

② Type 1 :

Type 1 known as context sensitive grammar. The CSG is used to represent context sensitive language.

Rule : ① CSG may have more than one symbol on the left hand side of their production rules.

- ② the number of symbol on left side not exceed the no. of symbols on right side.
- ③ $A \rightarrow \epsilon$ is not allowed unless A is a start symbol. It does not occur on right hand side of any rule.
- ④ the Type 1 production is in the form of $v \rightarrow T$ where, the count of symbol in v is less than or equal to T .

eg:

$$S \rightarrow AT$$

$$T \rightarrow xy$$

$$A \rightarrow a$$

③ Type 2

is known as context free grammar. Context free languages are languages which can be represented by the context free grammar (CFG). Type 2 should be type 1.

eg:

$$A \rightarrow a$$

where A is single non terminal & is any combination of terminals & non-terminals.

eg: $A \rightarrow aBb$

$$A \rightarrow b$$

$$B \rightarrow a$$

① Type 3

is known as Regular Grammar.
Regular languages which can be describe using regular expression.
These languages can be modelled by NFA or DFA.

— Type 3 is most restricted form of grammar. Type 3 grammar should be type 2 and type 1. It is in form of

$$V \rightarrow T^*V / \epsilon$$

eg : $A \rightarrow xy$

Assignment NO. 5.

① Construct a Turing machine for $\{wwR \mid w \in (a+b)^*\}$

② Explain with diagram models of TM

- i) multihed TM
- ii) Multitape TM
- iii) offline TM
- iv) Zero way initial tape.

③ Solve following pumping lemma p(n) of a non-negative integer given as

$$\sum_{i=0}^n i^3 = \left[\sum_{i=0}^n i \right]^2$$

④ Define properties of Recursive language & recursively enumerable language.

⑤ Construct Turing machine for $L = \{a^n 2^m 3^n \mid n \geq 1\}$